Ur Sandhai

AI for Bharat, Prototype Round Write-Up — Track: AgriTech — Individual entry

Live demo: https://sruthisureshkumar-arch.github.io/ursandhai/

Source code: https://github.com/sruthisureshkumar-arch/ursandhai

---

What This Prototype Demonstrates

Ur Sandhai, “our marketplace” in Tamil, turns hyperlocal crowd reporting into a same-day selling decision. Farmers within roughly ten kilometres of each other report what they're selling, at what price, and how strong demand feels; the app converts that into a recommendation, the best window to sell, the crop worth prioritising, and whether to hold back because a crop is already oversupplied nearby, delivered in Tamil and English. The build is a single self-contained HTML file: no backend, no build step, no installation. Open it and every mechanism described below is running within seconds.

What's New and Distinctive This Round

Three mechanics move past the standard average-price lookup and target the exact moment a farmer decides whether to sell. Fair Offer Check lets a farmer standing in front of a buyer enter the price on the table and receive an immediate verdict, benchmarked against the live local average, in English and Tamil, turning the project's founding problem into a tool a farmer can use in the moment rather than a line in a pitch deck. Sell Together watches for real oversupply among nearby crowd reports and surfaces a prompt to coordinate a shared sale, converting a penalty the scoring model already computes into something actionable. The price trend line is a genuine ordinary-least-squares regression, computed in the browser against whatever reports currently exist for a crop, showing whether the local price is rising, falling, or holding steady, with a percent-per-day figure: a lightweight, honest stand-in for the Prophet model planned for production, running on live data rather than a fixed number.

What's Operational Right Now

The geo-radius filter is a genuine Haversine distance calculation, written in plain JavaScript, holding every report to a ten-kilometre radius and a seventy-two hour window. Prices are bucketed into two-hour windows and averaged to surface the strongest-selling hours of the day. A weighted scoring formula combines average price, demand strength, and a supply-pressure penalty, so the system accounts for local oversupply rather than chasing the single highest price. The map is a live SVG rendering, built from the same coordinate math the scoring engine uses, not a static illustration. Every recommendation, verdict, and nudge is produced bilingually from the same underlying data, and when fewer than five reports exist nearby, the system falls back to a baseline mandi price instead of extrapolating from too little signal. Photographing a price board runs real optical character recognition through Tesseract.js, an open-source engine executing entirely client-side, no cloud round-trip involved. Submitted reports persist to local storage, so a demo session survives a page refresh, with a one-click reset to the seeded data. The layout is built to collapse to a single column under 760 pixels, since the people this is designed for use phones, not desktops.

What's Simulated This Round, and the Reasoning

Voice input and spoken output are stood in with free, browser-native APIs. Wiring billed cloud services into a five-day solo sprint, only to demonstrate a feature already achievable for free, was not where the time was best spent. Voice input uses the Web Speech API in place of Amazon Transcribe, listening in Tamil and transcribing locally, with the same parsing logic that would run against Transcribe's output extracting a price from the transcript. Spoken recommendations use the Web Speech Synthesis API in place of Amazon Polly. Everything downstream of those two inputs, the scoring, the map, the fair-offer verdicts, the sell-together logic, the trend regression, and the bilingual generation, is genuine and independent of any external service. One capability from the original proposal, identifying a crop from a photograph of the produce itself, as distinct from reading a price board, which now works, remains unbuilt. It's scoped as the next milestone rather than mocked, since the interface already collects the input this feature would consume.

Tools, APIs, and Datasets

Layer	In this build	Planned for production
Frontend	Plain HTML, CSS, and JavaScript, no framework or build tool	Same, or a lightweight framework if report volume warrants it
Design	Figma, used to establish the visual language before implementation	Same
Voice input	Web Speech API (SpeechRecognition), free and browser-native	Amazon Transcribe (Tamil and Hindi) within the AWS free tier
Text-to-speech	Web Speech Synthesis API, free and browser-native	Amazon Polly within the AWS free tier

OCR (price boards)	Tesseract.js, open-source, executing fully client-side	Amazon Textract within the AWS free tier
Vision (crop identification)	Not yet built	Amazon Rekognition
Forecasting	A genuine OLS linear regression on crowd reports, computed client-side	Prophet and scikit-learn, retrained nightly via a free GitHub Actions workflow
Government dataset	A compact baseline dictionary standing in for the live feed	Agmarknet mandi prices and arrivals, drawn from data.gov.in at no cost
Hosting	GitHub Pages, free static hosting	Same for the demo, plus a minimal backend once reports need to persist across visitors
Data storage	Browser local storage, surviving a page refresh	Supabase Postgres free tier, chosen for native geo queries
Fonts	Playfair Display and Inter, served from Google Fonts	Same

Feasibility, Cost Discipline, and a Path to Scale

Every component here runs on a free tier with no attached bill: GitHub Pages for hosting, free CDN delivery for fonts and libraries, browser-native speech APIs. Nothing paid is wired in, so nothing here can generate an unexpected invoice. The production architecture is designed to carry that discipline forward rather than abandon it once real cloud services enter the picture: AWS credentials would never touch the client, every AI call would run server-side behind authentication and rate limiting, IAM roles would be scoped to a single service each, and a hard budget alarm would cut access the moment spending moved past zero. Vision and speech workloads are scoped to migrate to self-hosted, open-source equivalents, Whisper, Tesseract, Coqui TTS, once usage outgrows the AWS free tier, keeping cost flat rather than letting it scale with adoption. That said, this is a design intention for a system not yet built, not a claim tested at any real scale. A realistic rollout begins with a handful of villages in a single district, using the existing crowd-report flow and the Agmarknet baseline as the cold-start fallback. Once a district sustains enough regular reporters to keep its data fresh, expansion proceeds district by district, since recommendation quality depends on reporting density, not installation count.

Social Impact

A farmer who cannot tell whether today's offer is fair is at a structural disadvantage against a buyer who can. A five to ten percent improvement in the timing of a sale, or in catching a lowball offer before accepting it, is a reasonable estimate of the stakes given typical day-to-day mandi price movement, though it remains an estimate: this prototype has not been piloted with real farmers, and no outcome data exists behind that figure yet. Sell Together is a narrower and more honest claim than a promise of collective bargaining power: it surfaces a timely suggestion when real oversupply is detected nearby. It does not yet give farmers a way to contact one another directly, so today it functions as a heads-up, not the coordination infrastructure an organised cooperative would provide.

Scope, Stated Plainly

This is a working demonstration of the central idea, hyperlocal crowd data converted into a same-day, bilingual selling decision, not a finished production system. The simulated components use free, zero-setup browser APIs specifically so the entire flow can be tested without an API key or a backend. The repository's README carries the same real-versus-simulated breakdown, alongside instructions for running the prototype locally.